Politechnika Łódzka

Projekt z przedmiotu: Uczenie Maszynowe

Temat: Generatywne Sieci Neuronowe

Rok akademicki: 2025/2026

Semestr zimowy

Dmytro Tynianskyi 253809

Michał Stasiak 251278

Aliaksander Ladutska 253193

1. Wstęp i cel

Uczenie maszynowe stanowi jeden z kluczowych obszarów współczesnej sztucznej inteligencji, umożliwiając tworzenie modeli zdolnych do samodzielnego odkrywania wzorców w danych. Szczególną uwagę w ostatnich latach poświęca się modelom generatywnym, których zadaniem jest nie tylko analiza danych, lecz także tworzenie nowych, realistycznych próbek na podstawie rozkładu danych uczących.

Jednym z najważniejszych i najczęściej badanych podejść w tej dziedzinie są Generatywne Sieci Neuronowe (Generative Adversarial Networks, GAN). Architektura GAN opiera się na rywalizacji dwóch sieci neuronowych – generatora oraz dyskryminatora. Generator stara się tworzyć dane jak najbardziej podobne do rzeczywistych, natomiast dyskryminator ma za zadanie odróżniać dane prawdziwe od wygenerowanych, co prowadzi do stopniowej poprawy jakości generowanych próbek.

Trenowanie sieci GAN jest procesem złożonym i niestabilnym, wymagającym odpowiedniego doboru architektury modeli, funkcji strat oraz hiperparametrów. W praktyce często stosuje się różne techniki stabilizacji treningu, takie jak odpowiednie funkcje kosztu, regularyzacja czy normalizacja wag, które pomagają ograniczyć problemy takie jak zanik gradientu czy niestabilna konwergencja.

W ramach niniejszego projektu zaprezentowano implementację sieci GAN z wykorzystaniem biblioteki TensorFlow/Keras. Projekt obejmuje definicję architektury generatora i dyskryminatora, zastosowanie technik wspomagających trening (m.in. spectral normalization oraz regulację jasności generowanych obrazów), a także mechanizmy zapisu checkpointów i wizualizacji postępów uczenia.

Celem projektu było zaprojektowanie, zaimplementowanie oraz przetestowanie generatywnej sieci neuronowej typu GAN, zdolnej do generowania realistycznych obrazów na podstawie losowego wektora wejściowego.

2. Zbiór danych

W projekcie wykorzystano zbiór danych zapisany w pliku, zawierający około **2800 obrazów medycznych przedstawiających prostatę**. Dane zapisano w postaci tablic NumPy, co umożliwia ich szybkie wczytywanie oraz bezpośrednie wykorzystanie w procesie uczenia generatywnej sieci neuronowej typu GAN. Wszystkie obrazy posiadają jednolity format oraz rozmiar, dzięki czemu mogły zostać łatwo przetworzone przez sieć neuronową. Każdy obraz reprezentowany jest jako macierz wartości pikseli, co zapewnia spójność danych wejściowych oraz stabilność procesu treningowego modelu. Przed rozpoczęciem uczenia dane zostały poddane normalizacji do zakresu odpowiedniego dla architektury generatora i dyskryminatora. Zbiór danych pełni rolę danych rzeczywistych, na podstawie których model uczy się odwzorowywać rozkład obrazów prostaty oraz generować nowe, realistyczne próbki.

3. Metody

Blok 1 — Already present (imports)

W pierwszym bloku zaimportowano niezbędne biblioteki do obliczeń numerycznych, przetwarzania danych oraz budowy i trenowania sieci neuronowych. Wykorzystano m.in. bibliotekę NumPy do operacji na tablicach, TensorFlow/Keras do definicji i uczenia modeli oraz Matplotlib do wizualizacji wyników. Dodatkowo uwzględniono możliwość użycia spectral normalization z biblioteki TensorFlow Addons w celu poprawy stabilności treningu dyskryminatora.

Blok 2 — Konfiguracja

Blok konfiguracji zawiera definicję kluczowych parametrów eksperymentu, takich jak rozmiar obrazów, wymiar wektora losowego (latent space), liczba epok, rozmiar batcha oraz współczynniki uczenia. Centralizacja parametrów umożliwia łatwą modyfikację ustawień eksperymentu oraz poprawia czytelność i powtarzalność badań.

Blok 3 — Checkpoint / logging

W tym bloku zaimplementowano mechanizmy zapisu stanu modeli w trakcie uczenia. Checkpointy umożliwiają zachowanie wag generatora i dyskryminatora oraz ich późniejsze odtworzenie.

Blok 4 — Training tricks / hyperparams

Zastosowano zestaw technik poprawiających stabilność uczenia sieci GAN. Obejmują one odpowiedni dobór hiperparametrów, takich jak learning rate, funkcje strat oraz sposób aktualizacji wag obu modeli. W projekcie uwzględniono również elementy regularyzacji i normalizacji.

Blok 5 — Pipeline danych

Blok ten odpowiada za przepływ danych od momentu wczytania do przekazania ich do modelu. Dane są przekształcane do odpowiedniego formatu tensorów oraz przygotowywane w sposób umożliwiający efektywne uczenie.

Blok 6 — Utility (general)

W tym bloku zdefiniowano funkcje pomocnicze wykorzystywane w różnych częściach projektu. Obejmują one m.in. funkcje inicjalizacji, narzędzia do wizualizacji oraz elementy ułatwiające zarządzanie eksperymentem. Rozdzielenie funkcji pomocniczych poprawia modularność kodu.

Blok 7 — Utility: BrightnessShift and SN wrapper

Zaimplementowano specjalne narzędzia wspierające proces uczenia. Funkcja BrightnessShift wprowadza losowe przesunięcia jasności obrazów, co działa jak forma augmentacji danych i zwiększa odporność dyskryminatora. Dodatkowo

zastosowano wrapper spectral normalization, który stabilizuje uczenie poprzez kontrolę normy wag w warstwach dyskryminatora.

Blok 8 — Modele: Generator i Dyskryminator

W tym bloku zdefiniowano architektury generatora i dyskryminatora. Generator przekształca losowy wektor wejściowy w obraz o zadanym rozmiarze, stopniowo zwiększając rozdzielczość danych. Dyskryminator analizuje obrazy i określa prawdopodobieństwo ich pochodzenia z danych rzeczywistych. Obie sieci zostały zaprojektowane w sposób umożliwiający efektywną rywalizację podczas treningu.

Blok 9 — Zapis siatki obrazów

W celu monitorowania jakości generowanych danych zaimplementowano mechanizm zapisu siatki obrazów w trakcie uczenia. Pozwala to na wizualną ocenę postępów generatora oraz obserwację zmian jakości i różnorodności generowanych obrazów w kolejnych epokach.

Blok 10 — Wczytywanie danych i przygotowanie datasetu

Blok ten odpowiada za wczytanie zbioru danych z pliku images.npy oraz jego wstępne przetwarzanie. Dane są normalizowane, konwertowane do odpowiedniego typu oraz przygotowywane jako obiekt datasetu TenorFlow, co umożliwia ich efektywne wykorzystanie w pętli treningowej.

Blok 11 — Budowanie modeli i optymalizery

W tym etapie następuje inicjalizacja generatora i dyskryminatora oraz przypisanie im odpowiednich optymalizatorów. Zastosowano osobne optymalizatory dla obu modeli, co pozwala na niezależną kontrolę procesu uczenia. Dobór parametrów optymalizacji ma kluczowe znaczenie dla stabilności i skuteczności treningu GAN.

Blok 12 — Pętla treningowa i zapis końcowy

Ostatni blok zawiera główną pętlę treningową, w której naprzemiennie aktualizowane są wagi generatora i dyskryminatora. W trakcie uczenia zapisywane są checkpointy oraz obrazy poglądowe. Po zakończeniu treningu następuje zapis końcowych wag modeli, umożliwiający dalszą analizę lub ponowne wykorzystanie wytrenowanej sieci.

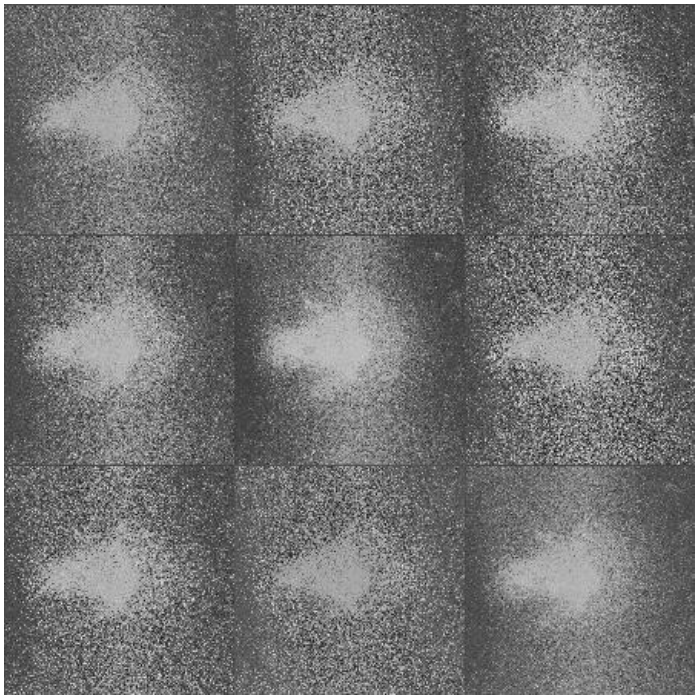
Generator — kluczowe funkcje

W generatorze zastosowano warstwy **Conv2DTranspose** do stopniowego zwiększania rozdzielczości obrazu. W warstwach pośrednich wykorzystano funkcję aktywacji **ReLU**, która zapewnia nieliniowość oraz stabilny przepływ gradientu podczas uczenia. W warstwie wyjściowej użyto funkcji **tanh**, ograniczającej wartości pikseli do zakresu $\langle -1, 1 \rangle$, zgodnego z normalizacją danych wejściowych.

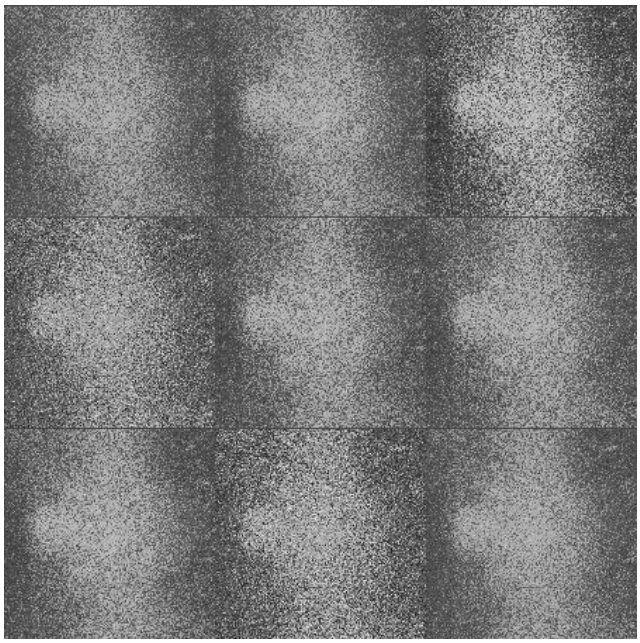
Dyskryminator — kluczowe funkcje

Dyskryminator opiera się na warstwach **Conv2D** odpowiedzialnych za ekstrakcję cech obrazu. Jako funkcję aktywacji w warstwach pośrednich zastosowano **Leaky ReLU**, co zapobiega problemowi martwych neuronów. Dodatkowo wykorzystano **spectral normalization**, stabilizującą proces uczenia.

4. Wyniki



Zdjęcie 1. Pierwsza próba na prostym modelu bez normalizacji danych (baza danych 72 zdjęcia)



Zdjęcie 2. Próba druga do sieci dodano wstępną normalizację danych i dodano 2 warstwy neuronów (baza danych 72 zdjęcia)

Zdjęcia 1 i 2 : Model na tym etapie uchwycił jedynie lokalizację gruczołu w centrum kadru. Widać "chmurę" punktów, która sugeruje, że sieć wie, gdzie szukać obiektu, ale nie rozumie jeszcze jego tekstury ani granic anatomicznych.

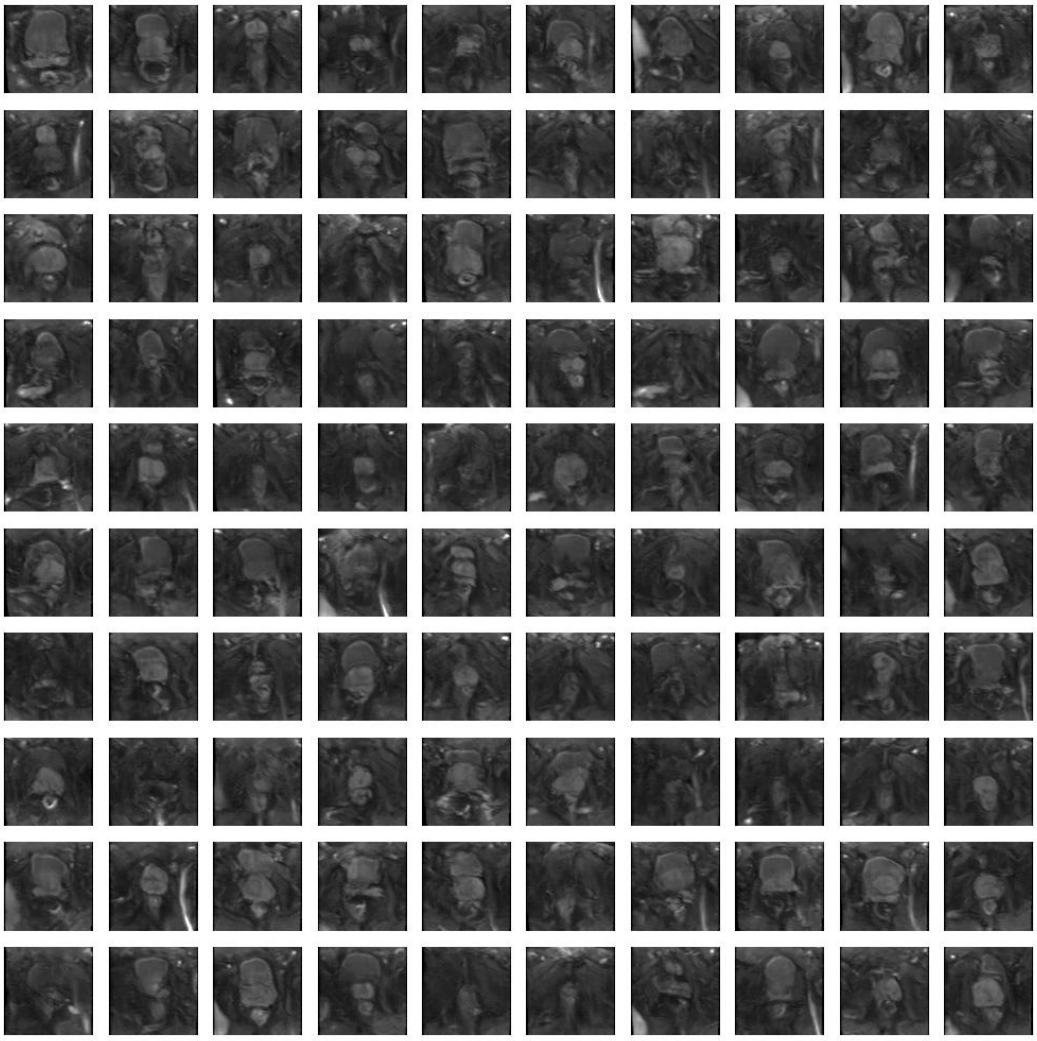
Zdjęcie 3. Model z większą ilością warstw i neuronów (baza danych więcej niż 2800 zdjęć)

Tutaj model zaczął generować przekroje osiowe (axial). Widoczne są zarysy strefy obwodowej i przejściowej prostaty oraz charakterystyczny cień kręgosłupa/tkanek kostnych z tyłu. Kształty są jednak nieco "rozmyte"

Zdjęcie 4. Finalny wynik uzyskany z 1200 epok uczenia na bazie 2800 zdjęć

To najbardziej zaawansowane wyniki. Kontrast między prostatą a otaczającymi tkankami jest znacznie wyższy. W niektórych klatkach (np. w drugim rzędzie) widać

bardzo realistyczne odwzorowanie strefy centralnej gruczołu



Zdjęcie 5. Zdjęcia wygenerowany na epoce 900

Metryki oceny zostały obliczone dla obrazów wygenerowanych z epoki 900 w porównaniu z próbką prawdziwych obrazów treningowych:

Średnie SSIM: 0.7344 (Structural Similarity Index Measure - Wskaźnik Podobieństwa Strukturalnego) - Ta wartość jest stosunkowo wysoka, co wskazuje na dobre podobieństwo strukturalne między obrazami wygenerowanymi a rzeczywistymi. SSIM równe 1 oznaczałoby identyczne obrazy.

Średnie PSNR: 26.5171 (Peak Signal-to-Noise Ratio - Szczytowy Stosunek Sygnału do Szumu) - PSNR jest zazwyczaj mierzone w decybelach (dB). Wartości powyżej 20 dB są ogólnie uważane za dobre dla jakości obrazu, zwłaszcza w modelach generatywnych, wskazując, że wygenerowane obrazy mają rozsądny stosunek sygnału do szumu w porównaniu z rzeczywistymi.

Średnie MSE: 0.0027 (Mean Squared Error - Średni Błąd Kwadratowy) - Ta bardzo niska wartość MSE sugeruje, że średnia kwadratowa różnica między wartościami pikseli obrazów wygenerowanych a rzeczywistych jest niewielka, co oznacza bliskie podobieństwo pod względem dokładności na poziomie pikseli.

Ogólnie rzecz biorąc, te metryki sugerują, że generator, w epoce 900, produkuje obrazy, które są dość podobne i mają przyzwoitą jakość w porównaniu z prawdziwymi danymi treningowymi.

5. Wnioski

Uzyskane wyniki wskazują na wyraźną ewolucję jakości generowanych obrazów prostaty – od początkowego szumu, poprzez stopniowe wyłanianie się struktur anatomicznych, aż do obrazów o wysokim kontraście i czytelnych granicach strefowych. Końcowa wersja modelu osiągnęła stabilność generowania, bez występowania zjawiska mode collapse, oraz wykazała zdolność do wiernego odwzorowania anatomii w przekroju osiowym, co sugeruje potencjalną przydatność diagnostyczną wygenerowanych danych. Głównym ograniczeniem projektu okazały się zasoby obliczeniowe – czas treningu przekraczający 4,5 godziny ogranicza dalszą optymalizację i testowanie bardziej złożonych architektur, dlatego dalszy rozwój projektu wymaga augmentacji danych, rozszerzenia modelu do postaci 3D, zastosowania transfer learningu oraz wykorzystania wydajniejszej infrastruktury obliczeniowej.

6. Udział w projekcie

Aliaksander Ladutska – wyszukiwanie kodów źródłowych, analiza gotowych rozwiązań, przygotowanie oraz wstępne przetwarzanie danych.

Dmytro Tynianskyi – pisanie części generatora i dyskryminatora, wprowadzanie ulepszeń modelu, poszukiwanie materiałów oraz metod stabilizacji procesu uczenia

.
Michał Stasiak – pisanie raportu, strukturyzowanie poszczególnych etapów projektu.

Wszyscy członkowie zespołu brali udział w uczeniu modelu. Do obliczeń wykorzystywano środowisko Google Colab, aby uzyskać dostęp do mocniejszych kart graficznych.